

P2956R1

Allow `std::simd` overloads for saturating operations

Published Proposal, 2025-05-19

This version:

<http://wg21.link/P2956R1>

Authors:

[Daniel Towner](#) (Intel)

[Ruslan Arutyunyan](#) (Intel)

Audience:

LEWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

Proposal to extend `std::simd` with overloads for saturating operations

Table of Contents

- 1 **Revision History**
- 2 **Motivation**
- 3 **Implementation Experience**
- 4 **Wording**
 - 4.1 Modify [`version.syn`]
 - 4.2 Modify [`simd.syn`]
 - 4.3 Add new section [`simd.saturating.math`]

References

§ 1. Revision History

R0 => R1

- Added implementation experience.
- Improved wording.

§ 2. Motivation

[P1928R4] introduced data parallel types to C++. It mostly provided operators which worked on or with `std::simd` types, but it also included overloads of useful functions from other parts of C++ (e.g., `sin`, `cos`, `abs`). Furthermore, in [P0543R3] a proposal is made to provide saturating operation support for some basic arithmetic operations and casts. In particular, `add_sat`, `sub_sat`, `mul_sat` and `div_sat` are provided. These perform saturating arithmetic operations which are effectively performed in infinite precision, and will return the smallest or largest value when it is too large to be represented in that type. In addition, `saturate_cast` is also provided to convert to a new type, and to saturate to the range of that type if required.

These saturating functions should be provided in `std::simd` as element-wise operations.

§ 3. Implementation Experience

The most common types of saturating operations are addition, subtraction, and casting. All three of these functions have been implemented in Intel's reference implementation and used in our software products. Where hardware support is available for a data type these functions compile into native instructions (e.g., 16-bit integer saturations compile into `vpaddsw`, `vpsubsw`, and `vpmovsdw` respectively). For data types which have no saturating support in the hardware for those three functions (e.g., large integers) the compiler can generate efficient code to perform the operation (in the case of LLVM the `builtin_add_sat` function is used to hand this task to the compiler, rather than having the library itself generate the required code sequence). Examples of native versus non-native instruction sequences are given here:

Source	Output from clang 20
<pre>// 16-bit saturating add // native instruction auto r16 = add_sat(x16, y16);</pre>	<pre>vpaddsw %zmm1, %zmm0, %zmm0</pre>
<pre>// 32-bit -> 16-bit saturating convert // Native instruction auto r16 = saturate_cast<int16_t>(x16);</pre>	<pre>vpmovsdw %zmm0, %ymm0</pre>
<pre>// 32-bit saturating add // Non-native (synthesised) auto r16 = add_sat(x32, y32);</pre>	<pre>vpadd %zmm1, %zmm0, %zmm2 vpcmpgtd %zmm2, %zmm0, %k0 vpmovd2m %zmm1, %k1 kxorw %k0, %k1, %k1 vpsrad \$31, %zmm2, %zmm0</pre>

The other saturating operations haven't been implemented in the reference software as they are rarely needed. However, they can be trivially implemented in terms of the existing draft C++26 support for scalar saturating operations, or an optimized equivalent can be synthesized.

§ 4. Wording

§ 4.1. Modify [version.syn]

In [version.syn] bump the `__cpp_lib_simd` version.

§ 4.2. Modify [simd.syn]

In the header `<simd>` synopsis - [simd.syn] - add at the end after the "Complex Math" functions.

```
template<simd-floating-point V>
    rebind_t<complex<typename V::value_type>, V> polar(const V& x, const V& y =

template<simd-complex V> constexpr V pow(const V& x, const V& y);

// [simd.saturating.math], saturating math functions
template<simd-type V> constexpr V add_sat(const V& x, const V& y) noexcept;
template<simd-type V> constexpr V sub_sat(const V& x, const V& y) noexcept;
template<simd-type V> constexpr V mul_sat(const V& x, const V& y) noexcept;
template<simd-type V> constexpr V div_sat(const V& x, const V& y) noexcept;
template<class U, simd-type V> constexpr rebind_t<U, V> saturate_cast(const V
```

Add the following to the end of the using declarations:

```
// See [simd.complex.math], simd complex math
using datapar::real;
using datapar::imag;
using datapar::arg;
using datapar::norm;
using datapar::conj;
using datapar::proj;
using datapar::polar;

// See [simd.saturating.math], saturating math functions
using datapar::add_sat;
using datapar::sub_sat;
using datapar::mul_sat;
using datapar::div_sat;
using datapar::saturate_cast;
```

§ 4.3. Add new section [simd.saturating.math]

Add the following section after [simd.complex.math].

basic_simd saturating math functions [simd.saturating.math]

```
template<simd-type V> constexpr V add_sat(const V& x, const V& y) noexcept;  
template<simd-type V> constexpr V sub_sat(const V& x, const V& y) noexcept;  
template<simd-type V> constexpr V mul_sat(const V& x, const V& y) noexcept;
```

Constraints:

The type `V::value_type` is a signed or unsigned integer type ([basic.fundamental]).

Returns:

A `basic_simd` object where the i^{th} element is initialized to the result of `sat-func(x[i], y[i])` for all i in the range `[0, V::size())`, where `sat-func` is the corresponding function from [numerics.sat.func].

```
template<simd-type V> constexpr V div_sat(const V& x, const V& y) noexcept;
```

Constraints:

The type `V::value_type` is a signed or unsigned integer type ([basic.fundamental]).

Preconditions:

For every i in the range `[0, V::size())`, `y[i] != 0` is true.

Returns:

A `basic_simd` object where the i^{th} element is initialized to the result of `sat_div(x[i], y[i])` for all i in the range `[0, V::size())`.

Remarks:

A function call expression that violates the precondition in the *Preconditions* element is not a core constant expression ([expr.const]).

```
template<class U, simd-type V>  
constexpr rebind_t<U, V> saturate_cast(const V& v) noexcept;
```

Constraints:

The types `U` and `V::value_type` are signed or unsigned integers ([basic.fundamental]).

Returns:

A `basic_simd` object where the i^{th} element is initialized to the result of `saturate_cast(v[i])` for all `i` in the range `[0, V::size())`.

§ References

§ Informative References

[P0543R3]

Jens Maurer. *Saturation arithmetic*. 19 July 2023. URL: <https://wg21.link/p0543r3>

[P1928R4]

Matthias Kretz. *std::simd - Merge data-parallel types from the Parallelism TS 2*. 19 May 2023. URL: <https://wg21.link/p1928r4>